

Milestone 2 DONE

EML Viewer Chrome Extension · Secure Rendering + Viewer Tab + Local Storage

In plain English

This is the big one. The extension now **actually shows you the email** — opening it in its own clean reader tab with the subject, sender, recipients and date up top and the email body below.

But the headline isn't "it displays email" — it's "it displays email **safely**." Emails can contain hostile code (hidden scripts, spy pixels that phone home, fake links). We built the email body to render inside a **locked glass box**: you can see and read everything, but nothing inside can run code, reach the internet, or touch the rest of your browser. We also made the extension **forget every email when you close Chrome**.

The "locked glass box" — how the safety actually works

The email body is shown inside a **sandboxed iframe** — a web feature that runs content with permissions stripped away. Think of it as a sealed display case. We deliberately leave *three* locks on and open *one small door*:

Setting	Effect (plain English)
No script permission	JavaScript in the email cannot run. A hidden <code><script></code> or an <code>onerror</code> trick simply does nothing.
No same-origin	The email is treated as a stranger from nowhere — it can't read your cookies, your storage, or reach into the extension.
No network (CSP + no permissions)	The extension asked for zero internet access , and the box has its own rule allowing only inline images. Tracking pixels can't load , so senders can't tell you opened the mail.
Popups allowed to escape 	The one open door: clicking a normal link opens it in a real new tab (with no back-reference to the email), so the viewer stays usable.

On top of that, before an email is ever stored we run it through a **sanitizer** that strips out scripts, click-handlers, and `javascript:` links. That's a second layer — a belt to go with the sandbox's suspenders. The sandbox is what actually guarantees safety; the sanitizer just cleans up and reduces what the box ever has to contain.

Where emails live (and when they disappear)

When you open an email, the extension saves it on your machine in the browser's built-in database (**IndexedDB**) and opens the viewer tab pointing at it. Nothing leaves your computer.

The privacy promise: a small "registry" of open emails is kept in *session* storage, which Chrome erases the moment you fully quit the browser. The database itself isn't auto-erased by Chrome — so on every startup our background script **compares the database against the registry and deletes anything orphaned**. After a restart the registry is empty, so all previously-opened emails get wiped. There's also a 12-hour expiry as a backstop. This is what makes "emails vanish when you close Chrome" actually true rather than just a claim.

What we built this milestone

File	Role
sanitize-html.js	Zero-dependency cleaner: parses the HTML into a tree, removes scripts / event handlers / unsafe links, re-serializes. Never uses fragile text find-and-replace.
storage.js	Reads/writes emails + attachments to IndexedDB, with delete-everything-for-an-email support.
mail-session.js	The session registry of open emails (clears on browser close).
viewer_page	The reader tab: header block + the sandboxed body iframe.
service-worker.js	Now also runs the startup reconciliation / 12-hour prune.
import_page	Rewired: parse → sanitize → save → open the viewer tab.

Proof it works (automated)

- **11 unit tests pass.** The sanitizer is tested against scripts, `onerror` handlers, `javascript:` links, whitespace-obfuscated schemes (`java script:`), iframes/objects/embeds, and form actions — plus it confirms safe things (`https + cid: + data: images`) are preserved.
- **Build succeeds** and the critical `sandbox="allow-popups allow-popups-to-escape-sandbox"` attribute survives the build untouched.

The one thing automated tests *can't* prove is the browser actually enforcing the sandbox — that's a Chrome guarantee, and it's exactly what your manual check below confirms.

✓ Your check in Chrome (~3 minutes)

1. Go to `chrome://extensions` and click the **reload** icon on the EML Viewer (it picks up the new build). If you hadn't loaded it: **Load unpacked** → the `dist/` folder.
2. Click the toolbar icon → drop `samples/basic.eml`. **Expect:** a clean reader tab — subject, From/To/Cc/Date, and the formatted HTML body. Click the "example.com" link → it should open in a **new tab**.
3. Go back to the import tab (or reopen via the icon) and drop `samples/malicious.eml`. This is the security test. **Expect ALL of:**
 - **No** alert popup appears, and the tab title does **not** change to "PWNERD" (the script and `onerror` were blocked).
 - The tracking pixel shows as a **broken image** (it could not phone home).
 - The "normal link" **opens a new tab**; the "javascript: link" **does nothing**.
4. **Privacy check (optional but satisfying):** fully **quit Chrome**, reopen it, and click the extension icon. Your previously-opened emails should be gone (the reconciliation wiped them). *Note: this needs the extension to have been loaded before quitting; loading unpacked persists across restarts.*

If the malicious sample stays silent and the basic one reads nicely, Milestone 2 is confirmed. Next: **Milestone 3** — inline images (logos/signatures) and the attachment list.

Known gaps until next milestone

- **Inline images won't show yet.** Emails embed logos using `cid:` references; resolving those to viewable images is Milestone 3. Until then they'll appear broken — expected.
- **No attachment list yet** in the viewer (attachments *are* already being saved). That UI is Milestone 3; previews/downloads are Milestone 4.